

The Total-Work Cost of Bounded Retry Guarantees

ANONYMOUS AUTHOR(S)

Bounding retries can move computation under protection; reducing protected work can increase total work. We characterize this tradeoff for persistent whole-kernel jobs with specified completion and observation operations. With a supplied write bound, independent jobs admit a complete polynomial total/protected-work frontier for every retry allowance; sufficient retries extend it to all dependency DAGs. One budget-unaware policy attains the least total-work curve under optimal protection at every budget for independent jobs. A full-program domination proof supports finite Pareto synthesis elsewhere, while precedence and hidden comparisons yield exact boundaries. Counting positive guard-entry charges separates informed and unaware protection guarantees. Java wrappers realize the core operation contract. An actual Linux pathname traversal supplies a conditional single-job source-work bound and exposes the distinct cost of guard acquisition; a higher-call-cap comparator avoids guards in first-conflict cases. Selected Roslyn checks preserve behavior, with mixed timings. These results identify exact resource choices and the source/interface conditions limiting their interpretation.

CCS Concepts: • **Software and its engineering** → **Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, scheduling, online policies

1 Introduction

An optimistic update loop prepares outside protection, validates its snapshot and publishes if still valid. Repeated failure can be bounded by recomputing under protection. This fallback limits calls but may protect all remaining work. A cached callback can instead reuse preparation on a match and recompute only on a mismatch. It reduces protected work, but a mismatch now pays for both preparation and recomputation. What is the least total work needed to guarantee a given bound on protected work?

We study a finite DAG of persistent jobs. Each executes a fixed-work opaque kernel using its own immutable input and the exact outputs of completed predecessors. The interface supplies cheap conditional publication, fresh protected completion and cached completion. A writer may change live inputs; a completed job stays completed. The resources are counted calls Q , all kernel work W , and its protected part L . The guarantees exclude waiting and do not imply a latency SLA.

Take works 8 and 16 with two completing calls. At supplied $B = 1$, the order $8 \rightarrow 16$ admits worst $(W, L) = (32, 16)$; reversing it requires $(40, 16)$ at the same protection cap. Requiring optimal protection at every hidden budget also requires $(40, 16)$ in both orders. All-fresh has $(24, 24)$. Figure 1 separates these contracts; the coordinate maxima need not coexist.

Dependency	Supplied $B = 1, L \leq 16$		One optimal- L curve for all B	
	Least worst W	Worst L	Least worst W at 1	Worst L at 1
$8 \rightarrow 16$	32	16	40	16
$16 \rightarrow 8$	40	16	40	16
Independent	32	16	40	16

Fig. 1. Same works and $Q = 2$; different information/optimality contracts. All-fresh has $(W, L) = (24, 24)$ and does not meet the displayed protection cap. Values follow from the whole-kernel model, not timings.

We derive exact supplied-budget frontiers for independent jobs and enough-retry DAGs, and one unaware independent-job policy with least total work under simultaneous optimal protection. Precedence and hidden comparisons give exact boundaries. A domination theorem makes

immediate-preparation trees complete beyond the polynomial regions. All lower bounds retain free inspections, old snapshots and charged retained preparations.

Fallback and runtime synchronization choice are established [5, 40, 41]. Scheduling with testing balances testing against execution costs [34, 37]; Pareto backward induction and budgeted robust games are inherited [11, 14]. Our claims concern exact joint resource values and the reduction from the specified completion-program class. Section 10 gives the explicit testing correspondence and distinguishes its objective and uncertainty budget.

Java checks the core contract. Linux supplies a single-job upper refinement and distinguishes protected computation from lock acquisition. Positive entry charges also change the theoretical information contract. Roslyn preservation checks have mixed timings; general latency improvement is unestablished.

2 Completion Interfaces and Resource Contracts

Write $\mathbb{N} = \{0, 1, \dots\}$. Jobs $J = \{1, \dots, n\}$, $n \geq 1$, form a finite DAG with predecessor set $\text{Pred}(i)$. Readiness requires all predecessors completed. Completion records an exact immutable output and persists despite later external writes. The objective is one successful foreground transformation per job, not a simultaneous snapshot or transaction. Policies are deterministic and causal; no randomized expected-cost guarantee is claimed.

Each ready job has a pure, opaque, whole kernel receiving its own-input identity and exact completed predecessor outputs. Every execution performs its fixed work $w_i > 0$, whether inside or outside protection. Partial/incremental evaluation, preparation before readiness and discharge by external work are excluded in this interface. A preparation is tied to its own-input identity and predecessor tuple. Captured outputs cannot be reconstructed by later reads of the mutable map.

States, histories and turns. A state $(x, \mathcal{M}, \mathcal{R}, \ell)$ contains live own-input identities, completed immutable outputs, a finite snapshot/record store, and program locals. A record carries its job, captured identity, predecessor tuple and computed result. Records may be retained, copied and selected, but not fabricated or retagged without computation. In Table 2, U means no selected record, C a selected possibly stale record, and D permanent completion. A cheap failure clears selection but preserves retained copies. At every finite history each unfinished job admits a legal identity never previously used for that job. Writes preserve completed outputs.

Visible histories list actions, returned values and completion events. Cached completion exposes its identity-comparison Boolean alongside the completed output; pure recomputation may return the same canonical result. Completion publishes its saved output over the live own-key entry. A program chooses a ready job/mode or an unprotected inspection, preparation or local action. The environment then selects a finite word of legal writes: the operation's *gate*, before comparison even within a callback. Job/mode is fixed before the gate; later record selection may use only the prior store. Completing comparison/computation/publication is indivisible to writes, and no guard survives return. Inspection honestly returns the live entry.

Preparation computes from a captured immutable own snapshot and exact completed parents; old raw snapshots are permitted. Its pure body has no live read or interaction. At completed-operation boundaries, contract it with its returned record and full work, replaying writes since capture before the next operation. Section 7's relation retains pending writes and does not place a result or full charge at an unfinished concrete prefix.

The common start precedes all computed initialization: every kernel execution contributes to W , including discarded preparations and results computed from supplied old raw snapshots. L counts its protected part. Q counts validation/publication/completion calls, including failures;

preparations and writers do not consume it. Waiting, common call overhead and nonkernel allocation are excluded. Neither clocks nor work counters are implicitly observed. Earlier call-charge and partial-repair objectives remain distinct in the supplement. Write $\Omega_k(T)$ for the sum of the largest $\min(k, |T|)$ works and $\Omega(T)$ for the total; omitted sets mean J , and $\Omega_0 = 0$. These aggregate/allocation objects are inherited [12, 13].

Table 1. Resources and information contracts used in the results.

Symbol	Meaning
$Q \leq n + r$	At most r noncompleting calls beyond the n mandatory completions.
W, L	All foreground kernel work and its protected part.
B	Upper bound on external writes; an input only for informed policies.
Ω, Ω_k	Total work and sum of the $\min(k, n)$ largest works.
$a_1 \leq \dots \leq a_n, A_k$	Sorted works and sum of the k smallest works, $A_0 = 0$.
$\mathcal{P}_3(B, r)$	Policies requiring the call cap only against at most B writes.
$\mathcal{U}_3(r)$	One budget-unaware policy requiring the cap at every finite budget.
$\mathcal{V}_r$	Policies in $\mathcal{U}_3(r)$ also attaining optimal L at every budget.

Table 2. Whole-kernel operations. Every validation or callback adds one to Q ; preparation adds none. Entries show successor state and added (W, L) . Each callback holds protection only within that operation.

Operation	Start	Match	Mismatch
Prepare	U	C, $(w_i, 0)$	—
Cheap publication	C	D, $(0, 0)$	U, $(0, 0)$
Fresh callback	U	D, (w_i, w_i)	—
Cached callback	C	D, $(0, 0)$	D, (w_i, w_i)

A cheap call publishes only on match and exposes success/failure. Conditional publication and a validation-only callback discarding its raw result implement this transition (Section 7). Two modes offer cheap/fresh operations; three add cached completion. The label “cheap” concerns kernel work. Failed validation leaves the entry unchanged. Free inspections/local record operations neither publish nor retain a guard. Direct acquire/release, retained protection, cross-job callbacks and outside completion are excluded, so this is not an arbitrary-lock-API lower bound. Selected operations must terminate. Infinite plays, including endless free actions, have infinite objective.

Strategies and admissibility. An informed strategy is a deterministic function $P(B, h)$; an unaware strategy is $P(h)$. Let $\mathcal{E}_b$ contain gate strategies using at most b writes along every play. For mode family j , $\mathcal{P}_j(B, r)$ consists of strategies completing with $Q \leq n + r$ against every $E \in \mathcal{E}_B$. By contrast, $\mathcal{U}_j(r)$ consists of one B -independent strategy satisfying that cap for every $b \geq 0$ and $E \in \mathcal{E}_b$. Every finite-write play against a fixed program is preserved by capping its environment at that play’s write count. Worst cost at budget B is $\sup_{E \in \mathcal{E}_B} L(P, E)$; the two minima use different admissible classes.

LEMMA 1 (FRESH-WRITE AND FINITE-ADVERSARY FACTS). *In Section 2’s game, a fresh own-key write at a selected i -operation’s gate invalidates every retained i -record. Any concrete prefix has both a no-write and a fresh-write continuation whenever that extra write respects the environment’s budget. An adversary bounded by b writes on every play forces any $P \in \mathcal{P}_j(b, r)$ to finish every job, regardless of free inspections or record retention.*

PROOF. A fresh identity differs from every retained target identity. Copying/selection cannot change that, and all observations and job/mode choices agree until the gate. The bounded adversary belongs to $\mathcal{E}_b$; leaving any job unfinished violates admissibility even with infinitely many free actions. Thus every job completes through Table 2. No unbounded adversary is truncated. $\square$

LEMMA 2 (WORK CHARGED BY USEFUL COMPARISON WRITES). *From the charged common start, consider a complete execution in which the adversary writes only fresh own-key identities at comparisons whose zero-write continuation would match. Let H be the multiset of resulting cheap failures and D the set of resulting cached mismatches. Then*

$$W \geq \Omega + \sum_{h \in H} w_{i(h)} + \sum_{i \in D} w_i.$$

Already-stale failures may consume calls, but need no extra charge in this inequality.

PROOF. Charge the computed record selected by the zero-write continuation, even if the actual failure branch selects an older one. The fresh write invalidates every prior target record. Before completion, neither failed cheap calls nor other foreground actions restore an earlier live identity; the restricted adversary never restores one. Each later useful comparison therefore charges a different computed identity. Final completion needs a further distinct execution: protected work or a matching preparation for the final identity. A cached mismatch pays both its pre-gate preparation and recomputation. Different jobs cannot share charges. This proves the bound with old snapshots, early computation and retained records. The charge is global; a suffix may have prepaid records. $\square$

3 Universal Call and Protected-Work Guarantees

For unfinished jobs S , let $\mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ denote the ready jobs.

THEOREM 1 (PROTECTED WORK UNDER A CALL CAP). *In Section 2's interface, fix integers $B, r \geq 0$. Among programs supplied with B that complete all jobs with $Q \leq n + r$ in every environment with at most B writes, the minimum worst protected work is*

$$L_3^*(B, r) = \Omega_{\min((B-r)_+, n)}, \quad L_2^*(B, r) = \begin{cases} 0, & B \leq r, \\ \Omega, & B > r, \end{cases}$$

with and without cached completion, respectively. Any DAG is permitted.

PROOF. At most r calls may fail to complete. Freshly prepared cheap attempts have disjoint read-to-comparison intervals; each failure needs a distinct write. If $B \leq r$ they give $Q \leq n + B$, $L = 0$. Otherwise switch after r failures to cached completion: at most $B - r$ writes remain, giving the stated bound. All-protected gives the two-mode upper bound.

For two modes when $B > r$, reject cheap calls with fresh writes, stopping unconditionally after $r + 1 \leq B$ rejections. The call cap forbids exhausting this budget, so every job completes under protection.

For three modes, put $k = \min(B - r, n) > 0$ and fix the k heaviest jobs as targets. The adversary maintains f , the number of noncompleting target comparisons, and p , the number of completed targets. Initially both are zero. It writes only at a target's comparison gate, with a fresh identity, while $f \leq r$ and $p < k$. It stops all writes permanently when $f = r + 1$ or $p = k$. Counters follow actual outcomes: f increases on a noncompleting target comparison and p on every target completion. While writing is active, cheap target calls fail and cached ones recompute. Fresh completion protects w_i without a write. After stopping, calls follow their ordinary semantics.

Let v be the writes so far. At each program node, after the preceding operation returns, induction gives $v \leq f + p$: each write led to a failure or a cached completion; fresh completion only increases p . Immediately before any further write, $f \leq r$ and $p \leq k - 1$, giving $v + 1 \leq f + p + 1 \leq r + k \leq B$. Once stopped, no write is added. Thus this is unconditionally an $\mathcal{E}_B$ strategy, including plays that violate the call cap.

For an admissible program, $f = r + 1$ is impossible: completion requires n completing calls in addition to those failures, already exceeding $n + r$. Thus $f \leq r$ throughout its play. Lemma 1 forces every target eventually to complete; each such completion is fresh or is a cached mismatch, both protecting its whole work. Consequently $p = k$ and $L \geq \Omega_k$. Free inspections, stale selections and retained copies cannot evade either case. Predecessors require their own calls, already included in n ; target predecessors are counted once in p . Dependencies delay targets without altering the accounting. $\square$

THEOREM 2 (ONE POLICY FOR EVERY WRITER BUDGET). *Let $\mathcal{U}_j(r)$ contain the j -mode programs that receive no B and complete with $Q \leq n + r$ in every finite-write environment. Set $U_j(B, r) = \inf_{P \in \mathcal{U}_j(r)} \sup_{\# \text{writes} \leq B} L(P, E)$. Then*

$$U_3(B, r) = L_3^*(B, r), \quad U_2(B, r) = \Omega \mathbf{1}[B \geq r].$$

One policy per mode family attains these values simultaneously for all B and positive works on any DAG, using only readiness and a global cheap-failure counter.

PROOF. Switch after r cheap failures to a fresh read/preparation with cached completion (three) or to fresh protected completion (two). This uses at most r extra calls universally. At $B < r$ no protection is needed; at $B = r$ cached completions after switching still match. Theorem 1 supplies the remaining upper and three-mode lower bounds.

For two modes when $B \geq r$, reject every cheap call with a fresh own-key write until r rejections, then stop. Only protected calls complete before this. Afterwards, another rejectable call could fail in a different finite environment, violating the call cap. Thus all jobs complete under protection with at most $r \leq B$ writes. That additional hypothetical write is absent from the lower-bound execution. Retained preparations and inspections cannot prevent a fresh write before comparison. $\square$

For equal works and $B > r$, the ratio is $n/\min(B - r, n)$. At $B = r$, only an informed two-mode program knows no write remains and may risk another cheap call. The unaware class must also be safe in extensions with another write, causing the boundary shift. A deployment that supplies a certified bound and requires the cap only within it instead uses $\mathcal{P}_j(B, r)$; that is a different information contract.

Per-update and shared-batch call allowances are different contracts. The supplement preserves the full combined local/shared formulas and proofs. Source comparisons below report their distinct caps; a lower observed protection count under a looser call cap is not a same-contract improvement.

4 The Total Work Needed for Optimal Protection

The protection optimum alone permits wasteful preparation. We minimize total work under simultaneous and supplied-budget protection contracts, for arbitrary r . The separate suprema ($\sup W, \sup L$) may have different witnesses; they do not determine $\sup(W + \lambda L)$. All computed initialization remains charged.

4.1 One Policy with Optimal Curves at Every Budget

Let $\mathcal{V}_r$ contain the policies in $\mathcal{U}_3(r)$ satisfying $L(P, E) \leq \Omega_{(b-r)_+}$ for every $b \geq 0$ and $E \in \mathcal{E}_b$. This requires one *simultaneous optimal-protection curve*; it is stronger than universal completion. Write $W_P(B) = \sup_{E \in \mathcal{E}_B} W(P, E)$.

LEMMA 3 (CAPPING A REALIZED PLAY). For $P \in \mathcal{V}_r$, a finite prefix containing d actual writes has protected work at most $\Omega_{(d-r)_+}$.

PROOF. Continue the prefix without writes. Admissibility forces completion. Cap that environment unconditionally at d : induction preserves each action and realized write, and there are no later writes to remove. The resulting environment belongs to $\mathcal{E}_d$ and preserves the complete play. Its total protected work satisfies the bound; nonnegative costs give the prefix bound. The program need not observe d . $\square$

THEOREM 3 (LEAST TOTAL-WORK CURVE UNDER SIMULTANEOUS OPTIMAL PROTECTION). For independent jobs with sorted positive works $a_1 \leq \dots \leq a_n$,

$$\inf_{P \in \mathcal{V}_r} W_P(B) = F_r(B) := \begin{cases} \Omega + Ba_n, & B \leq r, \\ \Omega + \max_{1 \leq m \leq \min(B-r, n)} \{ra_{n-m+1} + \Omega_m\}, & B > r. \end{cases}$$

One policy attains this entire curve: process jobs in ascending work order, freshly prepare before each cheap call until r cheap failures, and freshly prepare before each cached completion thereafter. It observes neither B nor cached comparison results. Sorting, prefix sums and prefix maxima construct the curve in $O(n \log n)$ arithmetic/comparison operations, representing its initial linear segment analytically.

PROOF. *Upper bound.* Each bad comparison needs a write in its distinct preparation-to-comparison interval. There are at most r cheap failures; cached mismatches occur only thereafter, at distinct jobs. Thus $Q \leq n + r$ and $L \leq \Omega_{(B-r)_+}$ at every budget. When $B \leq r$, at most B preparations are lost, each costing at most a_n .

For $B > r$, suppose cached mode starts at rank j and has $m \geq 1$ mismatches. All failed preparations were at ranks at most j and the m mismatched jobs lie in the suffix, so $j \leq n - m + 1$. Duplicate work is at most $ra_j + \Omega_m \leq ra_{n-m+1} + \Omega_m$, with $m \leq B - r$. Without a mismatch it is at most ra_n , below the $m = 1$ term. For each permitted m , match the first $n - m$ jobs, fail cheap r times at the next job, and mismatch all m cached completions. This attains its term using $r + m$ writes. The analogous maximum-job witness attains $\Omega + Ba_n$ at $B \leq r$.

Lower bound for arbitrary programs. Fix $m \leq \min(B - r, n)$ and target the m heaviest jobs. Write a fresh own-key identity only at target comparisons whose zero-write continuation would match. Let f count these useful cheap failures, c useful cached mismatches and t all target completions. Stop forever after $f = r + 1$ or $t = m$. At operation boundaries the write count is $d = f + c$, with $c \leq t$. Before another write, $d + 1 \leq f + t + 1 \leq r + m$. This is a globally $(r + m)$ -bounded adversary even on inadmissible programs.

An admissible program cannot reach $f = r + 1$, and every target eventually completes. Before all targets finish, no target comparison can match: a useful comparison is invalidated, and a stale one already fails or protects. Thus all targets complete protectively and $L \geq \Omega_m$. Lemma 3, applied at the realized final count d , gives

$$\Omega_m \leq L \leq \Omega_{(d-r)_+}, \quad d = f + c \leq r + m.$$

Positivity forces $d = r + m$, hence $f = r$ and $c = m$. All target completions are useful cached mismatches; this is a consequence of the contract, not an assumed normal form. Lemma 2 now charges r distinct failed preparations of work at least a_{n-m+1} , all m duplicate target kernels and mandatory Ω . Maximizing over m gives the lower bound.

Early budgets. The $m = 1$ witness just constructed has r useful cheap failures at a maximum-work job, then its useful cached mismatch. Truncate after the first $B \leq r$ writes. Its unchanged prefix has destroyed B distinct paid preparations of that unfinished job. Eventual completion needs another

distinct kernel, giving $\Omega + Ba_n$ by global charging. At $B = 0$ mandatory work suffices. Lower bounds include retained records, free inspections and paid early preparations after readiness. $\square$

The maximizing m need not be largest: works $(1, 100)$ and $r = 1$ give terms 200 and 102. Optimal W and optimal L may have different maximizing executions. The theorem identifies one least attainable pair of curves, not the work of every optimal- L policy. A relaxed protection requirement defines a different comparison.

Dependency boundary. The lower bound is order-independent. For any DAG, any topological cheap-then-cached policy attains $F_r(B) = \Omega + Bw_{\max}$ at $B \leq r + 1$, while retaining optimal L at all budgets. At $r = 0$, all-cached gives $F_0(B) = \Omega + \Omega_B$ for every DAG and budget. Later positive-retry values can differ: the chain $10 \rightarrow 1 \rightarrow 1$, $r = 1$, $B = 3$, requires $W = 33$, versus 32 for independent jobs. Two target writes force a cheap failure and cached mismatch at the 10-unit job. Its protected work already equals $\Omega_1 = 10$; the next job cannot be fresh on the no-more-write continuation, nor cheap with no retry left. A third write therefore duplicates that next unit, giving $30 + 2 + 1 = 33$. The chain policy attains this value. A least entire curve for arbitrary positive-retry DAGs remains unresolved.

4.2 The Complete Frontier for Independent Jobs

A supplied write bound permits a different policy for each resource contract. Here allow arbitrary retry slack $r \geq 0$. Sort independent jobs by positive work $a_1 \leq \dots \leq a_n$ and let $A_k = \sum_{j=1}^k a_j$, $A_0 = 0$. Put $h = \min(r, B)$ and $p = B - h = (B - r)_+$.

THEOREM 4 (INDEPENDENT-JOB FRONTIER). *For independent jobs, the exact $(\sup W, \sup L)$ frontier over $\mathcal{P}_3(B, r)$ is the nondominated subset of (Ω, Ω) and*

$$(\Omega + C_k, \Omega - A_k), \quad 1 \leq k \leq n - p,$$

$$C_k = ha_k + A_{k+p-1} - A_{k-1}.$$

For $p = 0$ the prefix difference is zero. If $p \geq n$ only (Ω, Ω) remains; at $B = 0$ the frontier is $(\Omega, 0)$. Sorting and a prefix-sum sweep construct the frontier in $O(n \log n)$ arithmetic/comparison operations.

PROOF. For selected k , process jobs in ascending work order. Count matching completions s and cheap failures f . Before each preparation, if $s = k$, finish the suffix fresh; otherwise prepare/cheap when $f < r$ and prepare/cached when $f = r$. Stop when all jobs finish. This policy reads no B after k is selected.

Let m count cached mismatches. Disjoint invalidation intervals give $f + m \leq B$; cached mode requires $f = r$, so $m \leq p$. If all jobs finished before $s = k$, then $n = s + m \leq k - 1 + p < n$, a contradiction. Hence exactly k matching jobs precede the fresh suffix. Their weight is at least A_k , giving $L \leq \Omega - A_k$, attained without writes.

Cheap failures occur before cached mode, at rank $s + 1 \leq k$, and cost at most ha_k . The j th cached mismatch has rank $s + j \leq k + j - 1$. Thus $W - \Omega \leq ha_k + \sum_{j=k}^{k+p-1} a_j = C_k$. To attain it, match the first $k - 1$ jobs, fail cheap h times on job k , then cause p cached mismatches at ranks $k, \dots, k + p - 1$. The next job $k + p \leq n$ matches and triggers fresh completion. For $p = 0$, job k itself matches after the failures. The call bound is $Q \leq n + h$ within budget and $Q \leq n + r$ against any finite number of writes.

For the lower bound, target ranks $k, \dots, n$. A globally B -bounded adversary writes only at target comparisons whose zero-write continuation would match. Already-stale cheap calls receive no write; stale cached calls protect without one. If the budget is not exhausted, every target completes protectively, giving $L \geq \Omega - A_{k-1}$. Otherwise let $f \leq h$ writes cause cheap failures; the other $B - f$

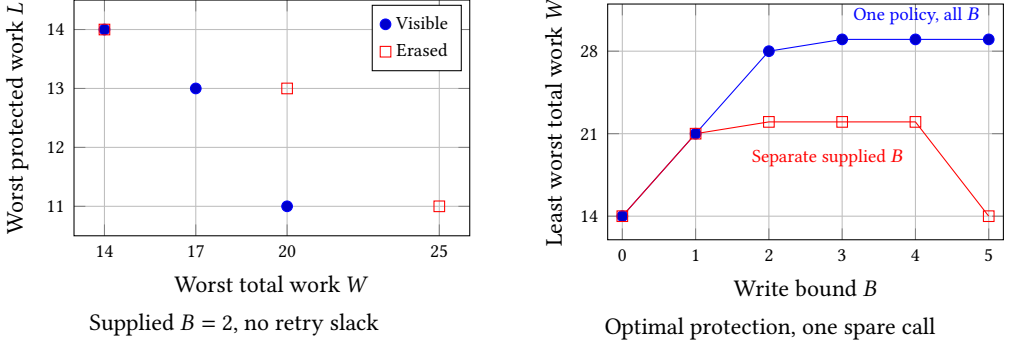


Fig. 2. Independent works (1, 2, 4, 7). Left: exact frontier points for two cached-observation contracts; no interpolation between points is implied. Right: at $r = 1$, both classes achieve $L^*(B) = (0, 0, 7, 11, 13, 14)$ for $B = 0, \dots, 5$, but the upper curve uses one policy meeting all bounds. The supplied-budget optimum can fall as B grows because its optimal- L cap also loosens; it compares different contracts. Values are kernel work, not timings.

cause cached mismatches at distinct targets. Lemma 2 gives

$$W - \Omega \geq f a_k + \sum_{j=k}^{k+B-f-1} a_j \geq C_k.$$

The actual distinct targets ensure this window exists. Increasing f replaces its last weight by a_k and cannot increase it; at $f = h$ it equals C_k . Wasted stale failures consume retry allowance without reducing this bound.

Consequently worst $L < \Omega - A_{k-1}$ forces worst $W \geq \Omega + C_k$. Since C_k is nondecreasing, a cap $M < C_1$ forces all work protected. Otherwise choose the largest k with $C_k \leq M$; the next index's target bound forces $L \geq \Omega - A_k$. At $k = n - p$, Theorem 1 supplies the same lower bound Ω_p . The attaining policies complete the frontier; nondominance removes tied thresholds. At $B = 0$, select $k = n$; at $p \geq n$, mandatory work and the protected-work lower bound give all-fresh. $\square$

One fixed r, k policy attains its stated pair for every $B \leq r + n - k$. At the next budget, matching $k - 1$ jobs, failing cheap r times and mismatching every remaining job gives $L = \Omega - A_{k-1} > \Omega - A_k$. Thus the constant protection guarantee's range is sharp. Selecting k still uses the supplied contract; this is not one optimal- L curve for all unknown budgets.

At $r = 0$, the optimal- L corner is

$$\min_{\substack{P \in \mathcal{P}_3(B, 0) \\ \sup L \leq \Omega_B}} \sup W = \begin{cases} \Omega + \Omega_{B+1} - w_{\max}, & B < n, \\ \Omega, & B \geq n. \end{cases}$$

Compared with Theorem 3, the extra total work required by the *simultaneous optimal- L contract* is $w_{\max} - w_{(B+1)}$ for $B < n$, using nonincreasing works, and Ω for $B \geq n$. This compares two specified policy classes, not unrestricted pointwise ignorance. Ties and $B = 0$ can eliminate the first gap.

4.3 When Retry Slack Covers the Write Budget

PROPOSITION 5 (AN ORDER-INDEPENDENT FRONTIER WITHOUT THE CACHED FLAG). *If $r \geq B > 0$, on every DAG the least worst L under $W \leq \Omega + M$ is*

$$\sum_{i: Bw_i > M} w_i.$$

Its attaining policy needs only cheap success/failure, even under the output-only cached contract of Section 6.2.

PROOF. Choose the k smallest jobs for cheap calls and all others fresh; follow any topological order until B cheap failures, then prepare/cache every unfinished job. Duplicate work is at most Ba_k , and protected work at most $\Omega - A_k$. Repeated writes at a maximum chosen job and the no-write path attain the separate bounds. Theorem 4's target lower bound uses no execution order and specializes to $C_k = Ba_k$. Tied thresholds give the displayed cap form. $\square$

For the 8/16 example, one spare call and supplied $B = 1$ permit $(W, L) = (32, 16)$ and $(40, 0)$ in either edge direction. Enough failures can reveal budget exhaustion through cheap results; the cached comparison need not be exposed. This does not eliminate the observation issue when $r < B$ or when one optimal- L curve is required at every unknown budget.

5 Precedence Determines the One-Write Frontier

DAG edges did not affect the protected-work value. They affect the least total work that realizes it. Consider a supplied $B = 1$ and a cap $W \leq \Omega + M$, $M \geq 0$. Define a scheduling instance by

$$(p_i, \tau_i) = \begin{cases} (w_i, 0), & w_i > M, \\ (0, w_i), & w_i \leq M. \end{cases}$$

Call the first jobs heavy and the others light. For a topological order π , $C_i(\pi)$ is the sum of p_j through job i .

THEOREM 6 (REDUCTION TO PRECEDENCE-CONSTRAINED SCHEDULING). *For arbitrary DAGs, $B = 1$ and $Q \leq n$, the minimum worst protected work subject to worst total work at most $\Omega + M$ is*

$$\ell(M) = \min_{\pi \text{ topological}} \max_i \{C_i(\pi) + \tau_i\}.$$

Lawler's backward minimum-cost sink rule [33] computes the order. Sweeping $M \in \{0, w_1, \dots, w_n\}$ and removing dominated pairs gives the entire W/L frontier.

PROOF. For an order π , complete heavy jobs fresh and light jobs by immediate prepare/cache until a mismatch. After that mismatch, prepare/cache the suffix. At most one job duplicates and its weight is at most M , so $W \leq \Omega + M$. A no-mismatch execution protects all heavy work. A mismatch at light i protects its earlier heavy prefix plus w_i , and nothing later. These cases give $\max_i (C_i + \tau_i)$.

For the lower bound, follow any admissible program's no-write execution. Every completing call is fresh or cached: a cheap call has a one-write failing continuation. Any matching cached call on i admits an own-gate fresh write that forces $W \geq \Omega + w_i$. Therefore every heavy job must be fresh. A cached call already stale without a write can be classified as fresh in this argument: it already protects w_i , and its wasted preparation only strengthens the work lower bound.

The no-write path forces at least its total fresh work in L . Each matching cached job i also gives a one-write witness with protected work at least its preceding fresh prefix plus w_i . Convert an optional fresh light job to cached in this lower-bound expression: later fresh prefixes decrease, and its new own term is at most the old total fresh work. Repeating cannot increase the expression.

The remaining fresh jobs are exactly the heavy ones, producing (p_i, τ_i) and the claimed minimum over topological orders. This proof covers arbitrary preparations and subsequent value-dependent choices; it does not assume the normal form of Section 6.

For completeness, the inherited scheduling exchange is short. Among sinks of the remaining graph, select j with minimum τ_j and place it last. In an optimal order with total remaining processing T and last job k , moving sink j to the end weakly decreases intervening completion times. Its new term $T + \tau_j$ is no larger than the old last term $T + \tau_k$, since k was also a sink. Repeat on the remaining graph. This works with zero processing times. Changing M matters only when a weight crosses the threshold; that job's own term is unchanged and later processing prefixes decrease. Hence $\ell(M)$ is nonincreasing, and the finite sweep is complete. $\square$

This is an equivalence from the completion-program resource constraint to a classical scheduling objective, not a new scheduling algorithm. A straightforward sink implementation takes $O(n^2 + |E|)$ per threshold and $O(n^3 + n|E|)$ for the sweep.

PROPOSITION 7 (WHEN THE OPTIMAL PROTECTION CAP PERMITS A SAVING). *At $B = 1$ and $L \leq w_{\max}$, the minimum worst total work is $\Omega + w_{(2)}$ if the maximum is unique and its job is a sink; otherwise it is $\Omega + w_{\max}$. Set $w_{(2)} = 0$ for one job.*

PROOF. If $M < w_{(2)}$, at least two heavy jobs have total exceeding $w_{\max}$. If the unique maximum has a successor, that successor's term exceeds $w_{\max}$ whenever $M < w_{\max}$. Tied maxima also force excessive heavy total below this threshold. Conversely, for a unique maximum sink, $M = w_{(2)}$ and placing that job last gives objective $w_{\max}$. At $M = w_{\max}$ all-cached always attains it. $\square$

For $8 \rightarrow 16$, cache the small job, then use fresh on match and cached on mismatch. The two paths cost $(24, 16)$ and $(32, 8)$, hence worst $(32, 16)$. Reversing the edge or imposing simultaneous optimal protection requires $(40, 16)$.

6 A Complete Finite Policy Class for Arbitrary Retry Slack

The preceding polynomial regions have direct lower bounds. For other DAG/budget combinations we justify finite policy search against the full class of programs with free inspections, early preparation after readiness, and retained records.

THEOREM 8 (RESOURCE DOMINATION BY IMMEDIATE-PREPARATION TREES). *For every $P \in \mathcal{P}_3(B, r)$ there is a finite fresh/cheap/cached tree T such that*

$$\forall E \in \mathcal{E}_B \exists E' \in \mathcal{E}_B : (Q, W, L)(T, E) \preceq (Q, W, L)(P, E').$$

Its depth is at most $n + r$, and it prepares only immediately before its selected cheap/cached node.

PROOF. Restrict the adversary to no writes except an optional fresh own-key write at a comparison whose zero-write continuation matches. Fix a concrete fresh identity at each branch history. A fresh call gives a unary fresh node. A matching cheap call gives completing and fresh-write-failing children; a matching cached call gives two completing children. An already-stale cached call becomes unary fresh, retaining its original zero-write continuation. Delete an already-stale cheap call and retain its failing continuation and fixed local choices in the witness history.

If the original prefix has u completions, h useful cheap failures and s deleted stale failures, its calls number $u + h + s$; the tree has $u + h$. Its unused failure allowance may increase, but need not be spent. All represented paths have at most $n + r$ nodes. Between them, no-more-write computation and deleted calls must terminate, or the original program would not complete in that finite-write environment. Finite branching and bounded depth therefore yield a finite tree.

Cap the restricted adversary at B and replace a zero-remaining-budget suffix by ready prepare/cached completions. Useful comparisons cost counted calls; the original cap forces completion along every represented path.

On a complete path let H be its binary cheap failures and D its binary cached-mismatch jobs. Lemma 2 lower-bounds original work by $\Omega + \sum_H w_i + \sum_D w_i$. Original protected work is at least the sum of the extracted fresh-node works and $\sum_D w_i$; a collapsed stale cached call already protects its kernel. A known-zero-budget suffix substitution can only lower protected work. Deleted stale cheap calls need no separately assigned preparation charge.

Realize the fixed tree using actual current inputs and exact saved parents, ignoring their values for scheduling. Prepare only at its selected cheap/cached node, never speculatively for a fresh node. Readiness follows the same completed set. Actual failures/mismatches have disjoint preparation-to-comparison intervals and inject into distinct actual writes; irrelevant or identity-reusing writes may remain invisible. Thus a path with $d = |H| + |D|$ bad comparisons requires at least d writes. Exactly d own-gate fresh writes realize that path and its original concrete witness. Define the witness adversary with an unconditional cap d , stopping after that many writes on every history. It is globally d -bounded, rather than merely spending d on one play.

The tree's exact path costs are $Q = n + |H|$, $W = \Omega + \sum_H w_i + \sum_D w_i$, and $L = \sum_{\text{fresh}} w_i + \sum_D w_i$. They are no greater than the original witness's. Any actual tree execution with at most B writes is therefore matched to an original environment in $\mathcal{E}_d \subseteq \mathcal{E}_B$. Correct outputs follow from the operations' semantics, not from reproducing the original program's concrete values under the same environment. $\square$

6.1 Exact Pareto Recurrence

Let $\mathcal{F}(S, b, a)$ contain nondominated suffix-tree bounds (e, l) , where S is unfinished, b is a conservative remaining write allowance, a remaining noncompletions, $e = \max W - \Omega(S)$ and $l = \max L$. Set

$$\mathcal{F}(\emptyset, b, a) = \mathcal{F}(S, 0, a) = \{(0, 0)\}.$$

For ready i , write $R = S \setminus \{i\}$ and take the nondominated union of

$$\begin{aligned} \text{fresh:} & \quad (e, w_i + l), \quad (e, l) \in \mathcal{F}(R, b, a); \\ \text{cached:} & \quad (\max\{e_0, w_i + e_1\}, \max\{l_0, w_i + l_1\}); \\ \text{cheap, } a > 0: & \quad (\max\{e_0, w_i + e_1\}, \max\{l_0, l_1\}). \end{aligned}$$

Both comparisons choose their matching child in $\mathcal{F}(R, b, a)$. Cached chooses its other child in $\mathcal{F}(R, b - 1, a)$; cheap chooses it in $\mathcal{F}(S, b - 1, a - 1)$. Children are selected independently.

LEMMA 4 (SUFFICIENT STATE FOR A SUFFIX TREE). *The recurrence gives exactly the nondominated immediate-preparation tree bounds. Here b counts the remaining allowed bad branches; it need not equal actual unobserved writes remaining.*

PROOF. A path with d bad branches needs at least d distinct invalidating writes. Conversely, exactly one fresh own-key write at each bad gate and no others realizes that path. Hence worst costs with at most b writes equal maxima over paths with at most b bad branches. Inert writes or multiple writes at one gate add no new path.

Induct on $|S| + a$. At $S = \emptyset$ there is no cost. At $b = 0$, prepare/cached completes every ready job with no duplicate or protected work, dominating other suffixes. Otherwise a fresh node has one child on R . A comparison has a matching child with allowance b and a bad child with allowance $b - 1$; a cheap failure additionally consumes one of a retries and leaves S unchanged. Each child decreases the induction rank. Readiness depends only on S ; under immediate preparation, costs depend only on the selected job/mode and outcome. These facts yield exactly the displayed charges.

The two children describe mutually exclusive observations. The controller may choose either continuation independently in advance; the adversary follows only one. It therefore receives no split budget across children. Taking a maximum separately in each resource is both necessary and sufficient for the two worst-case caps. Conversely every tree has precisely this root/child decomposition. Monotonicity of each charge/max expression allows a dominated child to be replaced by a dominating one. Emitted policies retain witness nodes; original histories with the same (S, b, a) need not be equivalent. Theorem 8 supplies full-program completeness. $\square$

Pareto backward induction itself is inherited.

There are at most $(r + 1)(\min(B, n + r) + 1)2^n$ states, with possibly exponential frontier sizes. The independent-job formula, enough-retry DAG formula and zero-retry one-write Lawler reduction identify polynomial regions; a general fixed-order theorem is not claimed.

The supplement also extracts one budget-independent tree for a universal program by retaining every branch up to depth $n + r$. Theorem 3 settles its least total-work curve for independent jobs; general DAGs may still require a Pareto set of curves.

6.2 The Frontier When the Comparison Is Hidden

Consider a weaker observation contract. Each pure kernel performs its prescribed work but returns the same immutable per-job object z_i on every invocation. Completion publishes z_i over the compared live entry. The cached primitive's *entire visible transition* exposes only that output and completion: it does not retain a compared current-input reference, Boolean, comparison-dependent record selection or callback-local side effect. Kernel internals, work counters, allocation IDs and clocks remain hidden. Free outside inspections and immutable old snapshots are still allowed. This condition is stronger than merely returning a constant: a callback that saves the current input elsewhere adds an observation.

THEOREM 9 (OUTPUT-ONLY RESOURCE FRONTIER). *Under this observation contract, the $r = 0$ frontier at a supplied B on any DAG is*

$$\text{ND}\{(\Omega + \Omega_B(C), \Omega(J \setminus C) + \Omega_B(C)) : C \subseteq J\}.$$

A fixed choice of cached jobs C and fresh jobs $J \setminus C$, executed in any topological order with immediate preparation, attains each candidate. At $L \leq \Omega_B$, least worst W is $\Omega + \Omega_B$ for $B < n$ and Ω for $B \geq n$.

PROOF. At $B \geq 1$, follow any program's no-write run. Cheap cannot be used: a fresh gate write would cause a forbidden extra call. Partition completions into matching cached calls C and fresh/already-stale cached calls F . For any $H \subseteq C$, $|H| \leq B$, insert one fresh write at each selected comparison gate. Each completion restores the same entire visible state by the erasure contract. Induction preserves all later inspections, record selections and modes. Lemma 2 gives $W \geq \Omega + \sum_H w_i$ and $L \geq \Omega(F) + \sum_H w_i$. The heaviest allowed H gives the stated partition pair. Immediate preparation attains it, because each mismatch needs a distinct write.

If $B < n$ and $F \neq \emptyset$, then $\Omega(F) + \Omega_B(C) > \Omega_B(J)$: the sum either includes all work or the B heaviest works plus a positive contribution. Hence the optimal- L cap requires all-cached. At $B = 0$, prepare/cache gives $(\Omega, 0)$; at $B \geq n$, all-fresh gives (Ω, Ω) . $\square$

For $1 \leq B < n$, all-fresh and the candidates caching the k lightest jobs suffice:

$$(\Omega + A_k - A_{k-B}, \Omega - A_{k-B}), \quad k = B + 1, \dots, n.$$

An exchange replacing a cached set by a prefix of the sorted works lowers its largest- B sum and increases the saved unprotected work; the full index argument is in the supplement. Sorting/filtering takes $O(n \log n)$ plus a topological order. This selects the cached set; edges do not change the values.

For independent $(1, 2)$ and $B = 1$, a visible comparison permits $(\max W, \max L) = (4, 2)$, but the output-only cap requires $(5, 2)$. More generally the Boolean reduces the optimal- L corner by $w_{\max} - w_{(B+1)}$ for $B < n$, and by zero for $B \geq n$. Ties and $B = 0$ can eliminate the saving. In the one-write DAG example, hiding the comparison makes both edge directions require $(40, 16)$. The native controls below instantiate the missing observation. Fresh output allocation with identical-reference return on match is another sufficient observation contract; purity alone supplies neither.

Hidden comparisons with positive retry slack. Theorem 3's independent-job policy ignores cached comparisons. On DAGs, erasure can cost work: the chain $2 \rightarrow 4 \rightarrow 1 \rightarrow 1$, $r = 1$, $B = 4$, has least universal $W = 17$ with visible comparisons and 18 without, requiring optimal L at every budget.

For visible attainment, use chain-order cheap-then-cached, except that a cheap failure and cached mismatch at the 4-unit job followed by a unit-job mismatch trigger fresh completion of the last unit. These events certify three writes and allow protection $4 + 1 + 1 = \Omega_2$. The worst work is 17. For erasure, fail cheap at the 4-unit job, then stop writing. Optimal $L(1) = 0$ and exhausted retries require matching cached completions on all residual jobs. Inserting one write at each preserves the visible state and duplicates $4 + 1 + 1$; mandatory 8 plus the failed preparation 4 gives 18. The supplement proves both all-program bounds and retains every witness; no general positive-retry erased frontier follows.

7 Actual External Writes in Java

Use Java 17 ConcurrentHashMap, one controller, permanent keys, deeply immutable non-null values, identity equality and never-reused own-key write identities. Kernels/writers preserve tags/shapes; kernels read only captured own inputs and saved completed parents. Key methods are stable; all methods terminate without side effects or recursive updates. Replacement may call equals [18, 19]. Normal operation progress is a premise.

Cheap publication wraps replace or validation-only compute, exposing only success/failure. Cached completion exposes its saved output and callback-local comparison Boolean, recomputing only on mismatch; fresh always computes. Policy observations retain opaque-reference equality, payloads, wrapper results, saved parents and selector state. Numeric IDs, clocks, counters, raw cheap-mismatch values and diagnostics are hidden. Reference renaming preserves aliasing.

State and operation correspondence. An injective map η covers live heap references, pending writes, saved results and available handles, preserving prior correspondences. Set $\mathcal{M}(i) = \eta(\text{done}[i])$ from the saved completing output; it never changes. The selectable store contains exactly available snapshots and computed records $(i, \eta(s), \mathcal{M}|_{\text{Pred}(i)}, \eta(z))$, excluding discarded ghost results. Native loops retain current handles; the larger program class requires actual retention. Locals map to ℓ .

Let Δ queue committed external assignments deferred in normalization. At boundaries,

$$\text{decode}_{\eta}(H.\text{map}) = \text{apply}(\Delta, x), \quad b_H = b_x + |\Delta|.$$

Here b counts actual publications, not pre-store diagnostic increments. Decoding follows forwarding nodes; Δ replays assignments, not callbacks.

Atomic compute invokes its callback once [17]. The pinned source [19] serializes same-key updates through checked monitors for list/tree remapping and publication; retrieval handles forwarding. Both wrappers return saved operation results, which may differ from newer live entries after protection ends. Completion records that saved output, never a reread. These actions implement Table 2's costs, store transitions and observations; source locators appear in the supplement.

THEOREM 10 (RESOURCE BOUNDS FOR ACTUAL WRITES). *Under these observation, wrapper, API and progress assumptions, Theorems 1–2's call/protected-work bounds and Sections 4–6's joint-work results*

hold for actual external writes under each result's specified observation contract, with separate or shared bins. Completed-operation traces preserve the listed results and Q, W, L ; this is not a mechanically verified JDK/JMM theorem.

PROOF. Group each completed pure kernel with its actual operands, result and work. Defer writes since capture in Δ and flush before the next operation, including inspection. Unfinished instructions are not zero-cost stutters. Post-publication/pre-return writes also enter Δ : saved results fix M while the map may be newer. Flush terminal queues without adding writes. Resize/tree conversion copies references; concurrent publications remain writes.

Upper policies inject bad comparisons into distinct committed writes through disjoint fresh preparation-to-comparison intervals. Progress transfers their call/work bounds.

For lower bounds, suspend after job/mode/store selection but before API entry. A terminating own-key writer installs a fresh reference without any foreground monitor held, even with colliding bins. It invalidates every prior target record. Thus bounded target adversaries preserve their write caps, mandatory completions and global work charges. Universal admissibility uses a further write in a different finite continuation. Exact saved parents preserve readiness; callback-local Booleans implement branches without diagnostic counters. These boundary arguments do not constitute a JMM proof.

□

Operational accounting. Earlier kernels perform $8w_i$ counted iterations; newer wrappers perform w_i units. Protected iterations and actual calls are counted; diagnostics are checker-only. Waiting, allocation, map traversal and writers are outside W, L . The finite matrix excludes concurrent resize.

8 Charging Guard Entry Changes the Contract

Charge each fresh/cached completion $g_i \geq 0$, even on a cached match, and cheap calls zero. Define $\Gamma = \sum_i g_i$ and $P_g = L + \sum_{\text{guarded } i} g_i$; guards still end at return.

PROPOSITION 11 (POSITIVE ENTRY CHARGES). *On every DAG, the optimal universal curve is*

$$U_g(B, r) = \Gamma \mathbf{1}[B \geq r] + \Omega_{(B-r)_+}.$$

At supplied $B = r$, informed optimum is 0, so $\Gamma > 0$ separates the information contracts.

PROOF. JIT cheap calls until r failures, then cached completions, attain the upper curve. For $B \geq r$, reject cheap calls before r failures and invalidate matching cached completions of the $k = \min(B - r, n)$ heaviest targets. Cap useful cheap writes at r , target writes at one each and total environment writes at $r + k$. Stale comparisons need none. After r failures, another cheap call could fail in a different finite continuation. Universal admissibility therefore forces every entry charge and all target bodies. Nonnegativity settles $B < r$; informed $B = r$ permits cheap-only completion. The supplement details the bounded adversary and informed one-write reduction; native charges and a joint W/P_g frontier are unclaimed.

□

9 Formal Corroboration and Operational Evaluation

RQ1 checks formulas and the operation contract; RQ2 tests a Linux upper refinement; RQ3 examines Roslyn preservation and cost limits. Fixed inputs precede execution; reanalyses are labelled post-outcome. All failures, timeouts, invalids and corrections remain archived.

An Apple M2/24 GiB runs Python 3.10.9/3.12.14 and OpenJDK 17.0.19; the cited map version was inspected, not rebuilt. Hashes bind inputs, code, commands and traces. Codex assisted design, checkers, sources, proofs, experiments, analysis and writing; Claude provided separate proof/direction

critiques and manuscript reviews. All measurements are archived author executions, without human studies or independent certification.

An input fixes graph, works and prices/footprints; a root adds budget/retries. Table 3 separates overlapping studies.

9.1 RQ1: Bounded Corroboration of the Proofs

Table 3. Bounded checks of the retained claims. The studies overlap in their source inputs; their denominators are reported separately. Finite agreement is not an all-program proof.

Obligation	Evidence and comparison
General call/ L values	Direct minimax versus formulas: 265,629 agreeing roots from 5,421 DAG/weight inputs.
General retry frontier	Immediate and retained-preparation finite models: 96,795 agreeing cases; $n \leq 4$, works $\{1, 2, 4\}$, $r = 0, 1, 2$.
Polynomial formulas	Retrospective: 1,998 independent and 32,526 enough-retry DAG cases (720 overlap). Prospective: 13,850 agreeing roots and 540,230 independently interpreted paths.
Unknown-budget curves	647 fixed independent roots and 34,814 interpreted paths; exact least curves agree, zero mismatches. Earlier 713 general roots remain.
DAG/observation boundaries	36 scope roots, zero mismatches; four erasure roots, 8,672 trees and 102,122 paths. All 8,668 protection violations retained.
Zero-retry regressions	26,844 Pareto cases; 18,390 scheduling cases; 2,004 independent corners; 237 simultaneous curves; 2,367 output-only cases. All agree with their stated comparators.
Native policies	Separate matrices of 2,076 zero-retry, 12,224 supplied-budget and 20,384 unknown-budget executions. Zero output/resource mismatches in each.
Flag-ignoring controls	Four of 52 violate L as predicted; all other 48 are retained.

The prospective general-retry study uses 461 multisets, $n \leq 6$, works $\{1, 2, 4, 7, 11\}$, $r \leq 3$ and $B \leq n + r$. All 9,240 first-excluded-budget controls violate L while preserving Q . Among output-only cases, 950 have positive Boolean value. These finite checks do not replace full-program proofs.

Actual writers exercise both wrappers, colliding/separate bins and canonical/allocating outputs. All 6,112 supplied-budget and 10,192 unaware wrapper pairs agree; 4,336/4,092 executions verify tree bins. The unaware selector sees no B , cached flag or diagnostics. All 512 groups/2,464 coordinates attain their curves; 12 corruptions reject. Unissued gates remain explicit. Entry-charge checks cover 26,874 scalar roots, 1,809 order cases and 86,376 paths without discrepancy.

Earlier aware/unaware records (9,314/17,714), six scaling timeouts, retained-guard/simple-rule counterexamples and cost fits successful in only two of eight cases remain distinct. The supplement retains the controlled Java blocking probe, all 24 timing cells and reversed disjoint-bin results; none establishes natural-arrival performance.

9.2 RQ2: A Native Traversal and Its Guard Boundary

Linux v6.12 `dentry_path_raw` first walks mutable `dentry` parents/names under RCU, validates `rename_lock`'s sequence, and on failure rebuilds under its reader-exclusive lock [6]. Pathname lock contention motivated the upstream optimistic fallback [8]. Our KUnit-only variant prepares a second traversal with a *new even* sequence, then acquires the lock and validates that saved sequence: reuse on match, reset/rebuild on mismatch. Initial `dentry/name` publication must precede parallel reading; subsequent relevant changes must participate in the global sequence, without wrap. This

covers the initialized `d_exchange` fixtures, not every filesystem: `d_mark_tmpfile` changes names without that sequence during initialization [7]. Mount traversal and `getcwd` are excluded.

Assume a private valid buffer with $1 \leq \text{capacity} \leq \text{INT_MAX}$, nonnegative observed lengths fitting `int`, valid dentry/RCU lifetimes, and successful stable-name copies. Let $C = \text{capacity} - 1$ after NUL. In the original factored loop, attempted names v , requested copy bytes c , fault-fill bytes f and successful slashes s satisfy $v \leq s + 1$, $f \leq c$ and $c + s \leq C$. Thus body events $T = 1 + v + c + f + s \leq 2C + 2$. For first/second validation failures X, Y , fresh even capture gives $X + XY \leq \min(B, 2)$ for actual global writer sections, including unrelated renames. Count one Q unit per lockless validation or guarded validation/completion block, not per native API call. The candidate has $Q \leq 2$, $T_{\text{total}} \leq (1 + \min(B, 2))(2C + 2)$ and $L_T = 0$ for $B \leq 1$. This is a single-job upper refinement: private-buffer return does not instantiate the full publication/observation contract. Guard/RCU instructions, waiting and postambles are separate; the capacity-4096 ceiling8192 is loose on short paths.

Actual ARM64 Linux boots in QEMU with two CPUs, lockdep and atomic-sleep checking. A pinned writer uses native `lock_rename/d_exchange`; endpoint gates are outside RCU and the mid-parent gate spins without sleeping. The fixed second matrix has eight path families, six capacities and eight schedules across five policies: 1,920 valid rows (948 paths/972 expected overflows), plus four detected defects. An independent fixture oracle checks full strings, offsets, canaries and post-return persistence against quiescent upstream. All rows and 2,878 body records agree offline, without lock/RCU diagnostics. A third execution brackets all 1,924 invocations with even nonwrapping global sequences: its 1,763 writer sections equal the designated counts in every row, and all prior output/body records agree. Returning unvalidated overflow, failing to restore the buffer or ignoring final validation breaks output. Reusing the first sequence preserves output but violates the protection bound at $B = 1$. No copy-fault/allocation failure is injected; native-time performance is unmeasured.

Table 4. Source event counts at the scripted gates. Initial path is `/driver/cache/entry/leaf`; first exchange moves it to `/alternate`, second restores it. G counts reader-exclusive acquisitions.

Policy	One write				Two writes			
	T	L_T	Q	G	T	L_T	Q	G
Upstream	41	12	2	1	58	29	2	1
Cached	41	0	2	1	70	29	2	1
Two cheap, then locked	41	0	2	0	70	29	3	1

All 384 cached/two-cheap pairs agree on output and body vectors; 138 first-invalid/second-valid pairs avoid the guard, while 144 two-failure pairs need an extra call. This is not dominance under equal universal Q caps. The source supplies no caller-required $Q = 2$ SLA. A stronger native name-snapshot comparator copies inline names or retains external-name references under protection, then materializes outside. Its allocation, dentry locks, capture steps and refcounts are separate from L_T ; five 64-slot overflows retain the discarded capture work before protected fallback. Its body-only zero is not zero total protected work or a lower bound for larger workspaces. All adverse results and the first 1,536-row matrix remain; the populations overlap.

9.3 RQ3: Preservation and Cost Limits in Roslyn

Roslyn's `Workspace.cs` at 6c4a46a3 transforms an immutable solution outside a semaphore, validates identity inside and retries [9, 10]. Our `SourceText`-only patch switches after r shared failures to fresh protected transformation (two modes) or prepared reuse with protected recomputation

on mismatch (three). A stronger rebase reuses document contents under explicit identity guards; even a successful reuse needs a protected merge. Lazy work, allocation, version generation and cache-dependent merge costs prevent fixed-work transfer. No upstream numerical cap or SLA was found.

All 236 authored cases agree on 2,190 events, 1,253 publications and 2,653 snapshots; 12 corruptions reject. A separate 96-process compiler study checks references, parse options and linked text with cold/warm caches: 2,556 project projections match the declared expectations and eight corruptions reject. The original same-text deadlock and four missing-document timeouts are retained; corrected patches pass the fixed 14 error cases and reentry controls. These are selected behavior checks using Roslyn on both sides, not general concurrent equivalence. Custom loaders, changed-text reentry and arbitrary compiler behavior remain outside scope.

Arrival tests compare shared and per-update caps, two and three modes, and both baselines. Eight balanced process blocks each serve 32 updates and 64 FIFO writers at 0, 10, 100, 1000 μ s. All 2,560 measured and 512 warm-up units pass source/count checks; 12 corruptions reject. At $r = 2$, local and shared caps permit 96 and 34 calls, respectively, precluding an equal-cap comparison.

Shared three/two foreground-time ratios at 0/10 μ s are 1.72 [1.40,2.11] and 1.58 [1.17,2.16], favoring two modes. Slower-arrival and all writer-response intervals include1. Geometric means use eight paired block ratios and descriptive 95% intervals from 10,000 fixed block-bootstrap samples, without multiplicity correction. Baseline ratios 0.83–1.08 all include1, leaving instrumentation overhead unidentified. Full mixed results remain in the supplement; authored arrivals, eight blocks and one host limit inference.

10 Related Work and Research Lineage

Fallback and contention management. Kung and Robinson [5], Section 3.3, retain a semaphore during reexecution after repeated failure. Bounded retries in lock elision and RPC are also established [25, 26, 30, 31]; HTM aborts need not consume writes [27], and RPC may terminate with failure. Welc et al. [32] preserve a transaction prefix during irrevocable transition. Attiya et al. [23] analyze unaware contention management with a makespan bound in per-job failures/resources; Guerraoui et al. [3] give bounded-delay progress and competitive makespan. Alistarh et al. [24] optimize abort timing given abort cost and unknown remaining duration. Runtime switching, hysteresis and measured retry-budget tuning predate ours [4, 40, 41]. These distinct progress/time objectives do not supply our exact persistent-job W/L contract; the supplement records source locators and limitations.

Synthesis and robust control. Černý et al. [14] choose retries, guards and caches under weighted limit-average safety; their erratum corrects complexity to PSPACE. Later work optimizes protected-statement counts and profiles contention [15], or synthesizes deadlock-free preemption-safe lock placement [16]. Global-budget and information-state minimax DP are inherited [11, 20, 28]. General games can encode finite instances of our problem. Our specialization proves resource domination of arbitrary retained-preparation programs by finite trees and exact resource values in polynomial regions. Neither backward induction nor Pareto composition is new.

Scheduling with testing. At $r = 0$, fresh completion is untested execution of cost $u_i = w_i$; cached completion tests for $t_i = w_i$ then executes for $p_i \in \{0, w_i\}$. Thus W counts testing plus execution and L execution alone. For a deterministic tree, a realized mismatch set can be fixed as hidden bad-job labels; conversely own-gate writes realize those labels. Dynamic writing alone does not distinguish this canonical game from testing.

Dürr et al. [34] introduce adversarial testing with binary outcomes; Buld and Schulz [37] and Albers and Eckl [38] allow unequal test times/upper bounds. Their objectives concern competitive

completion time or makespan. Damerius et al. [35] already separate testing expenditure and execution, with a *policy-side* testing budget. Our B instead limits adverse outcomes; even at $B = 0$, preparations remain allowed. Dufossé et al. [36] study adaptive testing/minimax trees; their oracle reveals durations even for untested execution, and their polynomial adaptive result assumes the two-phase conjecture. Albers and Janke [39] already bound failing jobs and sum largest adverse durations for irrevocable parallel-machine assignment. Our specialization returns causal conditional-completion policies for separate absolute W/L caps and a retry allowance. The informed comparator knows only B , not outcomes.

Precedence and reuse. Lawler’s algorithm and exchange argument [33] are used unchanged. Theorem 6 establishes the equivalence from completion programs under a total-work cap to that scheduling problem. Without a protection cap, all-fresh already minimizes W . Universal restart policies [42] concern independent Las Vegas runs and expected time. Build systems [1], self-adjusting computation [2] and demand-driven incrementality [29] reuse unaffected work. Such stronger operations can invalidate the whole-kernel premise; separate footprint-repair results remain in the supplement.

11 Limitations and Threats to Validity

The interface excludes replenishment, structural change, exceptions, external completion and retained guards. For an earlier scalar objective, retained guards improve 5/384 inputs (242 versus 243) and one intermediate state (45 to 43). These stronger-interface counterexamples do not evaluate the present W/L frontier.

Results require deterministic policies, positive fixed work and charged initialization. Polynomial regions cover independent jobs, enough-retry DAGs and the stated zero-retry cases. General synthesis may be exponential; least unaware curves remain unresolved for general positive-retry DAGs. Numeric IDs/timing change observability. Supplied B is a premise; Roslyn costs remain uncalibrated.

Instrumentation and cache/JIT effects limit timings; human benefit is unmeasured. Earlier scalar gains occur in 2/96 cells, cost fits in 2/8, and all-budget compilation can cost more than requested-value computation. In all 16 Valkey digest cells, cooperating writer-side maintenance beats reader repair in time/transfer, with mixed maximum command times. This stronger operation supplies no importance evidence here. Finite/AI-assisted checks are neither mechanical proofs nor independent reproduction.

12 Conclusion

Independent jobs admit exact joint frontiers and a least-work policy under optimal protection at every hidden budget. Precedence and observations define further boundaries; full-program domination supports finite synthesis elsewhere. Linux demonstrates a conditional source bound and the separate guard resource. Native multi-job minimax transfer and general performance benefit remain open.

Data Availability

The supplement [22] provides code, inputs, all outcomes/corrections, proofs and replay commands, preserving prior charged/partial-repair results and fixed163. Zero-fee evidence [21] is unchanged. Modified sources retain licenses; upstream archives, runtimes, articles and private correspondence are excluded.

References

- [1] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. <https://doi.org/10.1145/3236774>. Author version: <https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf>.
- [2] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. <https://doi.org/10.1145/1480881.1480907>. Author version: <https://ttic.uchicago.edu/~pl/sa-sml/pop109.pdf>.
- [3] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. <https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf>.
- [4] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. <https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues>. Errata.
- [5] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [6] Linux contributors. 2024. Linux v6.12, fs/d_path.c and fs/hostfs/hostfs_kern.c, commit adc218676eef25575469234709c2d87185ca223a. Accessed September 11, 2026. https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/d_path.c.
- [7] Linux contributors. 2024. Linux v6.12, fs/dcache.c: name snapshots, d_exchange, and tmpfile initialization. Accessed September 11, 2026. <https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/dcache.c>.
- [8] Waiman Long. 2013. dcache: Translating dentry into pathname without taking rename_lock. Linux commit 232d2d60aa5469bb097f55728f65146bd49c1d25. <https://github.com/torvalds/linux/commit/232d2d60aa5469bb097f55728f65146bd49c1d25>.
- [9] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. Workspace.cs, Solution.cs, SolutionCompilationState.cs and SolutionState.cs. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [10] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [11] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [12] Avi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [13] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [14] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [15] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [16] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [17] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [18] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [19] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [20] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [21] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e011f0. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbb422390ad8d29785ad0023f>.
- [22] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. General-retry, universal-curve, Linux-source and guard-entry supplement, commit 6ca1285. <https://github.com/research-artifact-archive/research-artifact/tree/6ca128573b758bf7706cbf8540237925435d6c2>.

- [23] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. <https://cs.idc.ac.il/~tami/Papers/podc06.pdf>.
- [24] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint* arXiv:1804.00947v1. <https://arxiv.org/abs/1804.00947>.
- [25] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision d2097651. `elision-lock.c` and `elision-conf.c`. <https://github.com/bminor/glibc/tree/d2097651cc57834dbfcaa102ddfacae0d86cfb66/sysdeps/unix/sysv/linux/x86>. Official tunables: https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html.
- [26] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision 54f20959. `c2_MacroAssembler_x86.cpp` and `globals_x86.hpp`. <https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cafa0defb956e13a2c/src/hotspot/cpu/x86>.
- [27] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. <https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html>.
- [28] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint* arXiv:2301.05089v2. <https://arxiv.org/abs/2301.05089>.
- [29] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. 2014. Adapton: Composable, Demand-Driven Incremental Computation. In *PLDI*. <https://doi.org/10.1145/2594291.2594324>. Author version: <https://www.cs.tufts.edu/~jfoster/papers/pldi14.pdf>.
- [30] Mike Ulrich. 2016. Addressing Cascading Failures. In *Site Reliability Engineering*, Chapter 22, Retries. <https://sre.google/sre-book/addressing-cascading-failures/>.
- [31] Envoy contributors. n.d. `CircuitBreakers.Thresholds.RetryBudget`. Accessed September 10, 2026. https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit_breaker.proto.html.
- [32] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. 2008. Irrevocable Transactions and their Applications. In *SPAA*, 285–296. <https://doi.org/10.1145/1378533.1378584>.
- [33] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>. Author technical report ERL-M327 (1971): <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1971/Archive/ERL-m-327.pdf>.
- [34] Christoph Dürr, Thomas Erlebach, Nicole Megow, and Julie Meißner. 2020. An Adversarial Model for Scheduling with Testing. Author preprint arXiv:1709.02592v3. <https://arxiv.org/abs/1709.02592v3>.
- [35] Christoph Damerius, Peter Kling, Minming Li, Chenyang Xu, and Ruilong Zhang. 2023. Scheduling with a Limited Testing Budget: Tight Results for the Offline and Oblivious Settings. Author preprint arXiv:2306.15597v1. <https://arxiv.org/abs/2306.15597v1>.
- [36] Fanny Dufossé, Christoph Dürr, Noël Nadal, Denis Trystram, and Óscar C. Vázquez. 2021. Scheduling with a Processing Time Oracle. Author preprint arXiv:2005.03394v3. <https://arxiv.org/abs/2005.03394v3>.
- [37] Felix Buld and Andreas S. Schulz. 2026. Scheduling with Testing: Competitive Algorithms for Minimizing the Total Weighted Completion Time in the Adversarial Model. Author preprint arXiv:2606.25166v1. <https://arxiv.org/abs/2606.25166v1>.
- [38] Susanne Albers and Alexander Eckl. 2020. Explorable Uncertainty in Scheduling with Non-Uniform Testing Times. Author preprint arXiv:2009.13316. <https://arxiv.org/abs/2009.13316>.
- [39] Susanne Albers and Maximilian Janke. 2021. Online Makespan Minimization with Budgeted Uncertainty. In *WADS*. https://doi.org/10.1007/978-3-030-83508-8_4. Author version: <https://maximilian-janke.de/data/BudgetedUncertainty.pdf>.
- [40] Beng-Hong Lim and Anant Agarwal. 1994. Reactive Synchronization Algorithms for Multiprocessors. In *ASPLOS VI*, 25–35. Author version: <https://groups.csail.mit.edu/cag/pub/papers/pdf/reactive.pdf>.
- [41] Takayuki Usui, Reimer Behrendts, Jacob Evans, and Yannis Smaragdakis. 2010. Adaptive Locks: Combining Transactions and Locks for Efficient Concurrency. *Journal of Parallel and Distributed Computing*. <https://doi.org/10.1016/j.jpdc.2010.02.006>. Author version: <https://people.cs.umass.edu/~yannis/adaptive-jpdc.pdf>.
- [42] Michael Luby, Alistair Sinclair, and David Zuckerman. 1993. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47, 4, 173–180. Author version: <https://www.cs.utexas.edu/~diz/pubs/speedup.pdf>.